Module 7

AVL Trees

AVLtrees:

- **Adelsion Velski and lendis** in 1962 introduced binary search tree structure that is balanced with respect to height of subtree
- The tree can be made balanced and because of this retrieval of any node can be done in O(log n) times where n is total number of nodes.
- From the name of the scientists ,the tree is called AVLtree.

Balanced factor:

Balance factor = **height of left subtree - height of right subtree** (OR) **height of right subtree - height of left subtree**.

For any node in AVLtree , the balanced factor BF(T) is -1,0,+1.

- *Most of the BST operations (e.g., search, max, min, insert, delete.. etc) take **O(h)** time where **h** is the height of the BST.*
- *The cost of these operations may become **O(n)** for a **skewed Binary tree**.*
- *If we make sure that the height of the tree remains **O(log(n))** after every insertion and deletion, then we can guarantee an upper bound of **O(log(n))** for all these operations.*
- *The height of an AVL tree is always **O(log(n))** where **n** is the number of nodes in the tree.*

Advantages & Disadvantages of AVL Trees

Advantages

•The height of the **AVL tree** is always balanced. The height never grows beyond log N, where N is the total number of nodes in the **tree**.

•It gives better search time complexity when compared to simple Binary Search **trees**.

•**AVL trees** have self-balancing capabilities

Disadvantages

- Slow Inserts and Deletes.
- The largest **disadvantage** to using an **AVL tree** is the fact that in the event that it is slightly unbalanced it can severely impact the amount of time that it takes to perform inserts and deletes.
- Fast Updating Systems.

TYPES OF ROTATION

Single Rotation

Case 1: Right right rotation (RR Rotation)

Case 2: Left left rotation (LL Rotation)

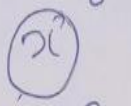
Double rotation

Case 3: Right left rotation (RL Rotation)

Case 4: Left right rotation (LR Rotation)

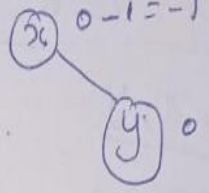
Case 1: x, y, z

Insert x:

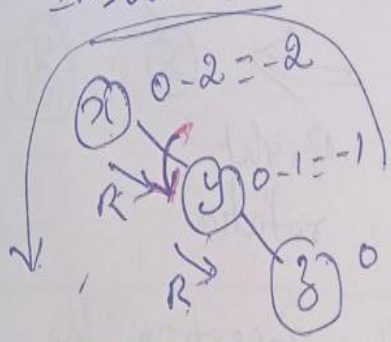


RR

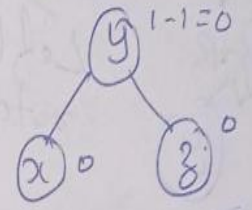
Insert y



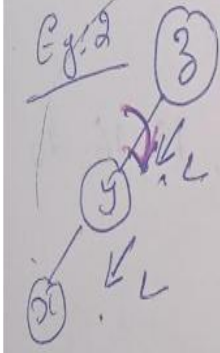
Insert z:



Apply Left rotation.

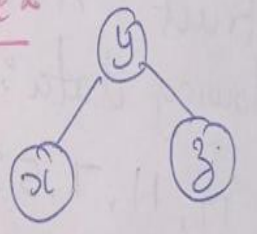


Eg:2

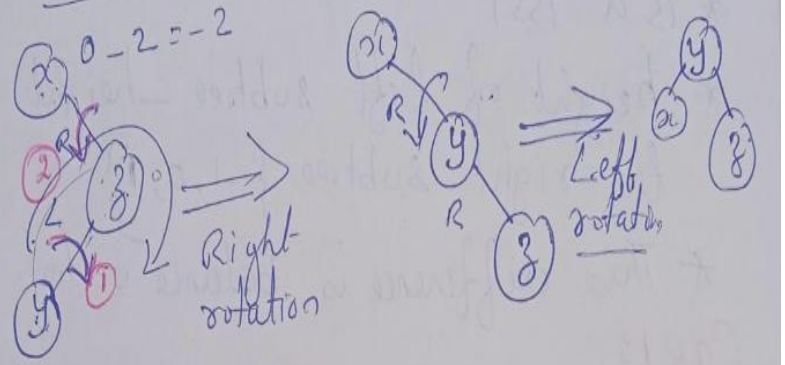


LL Case 2

right rotation

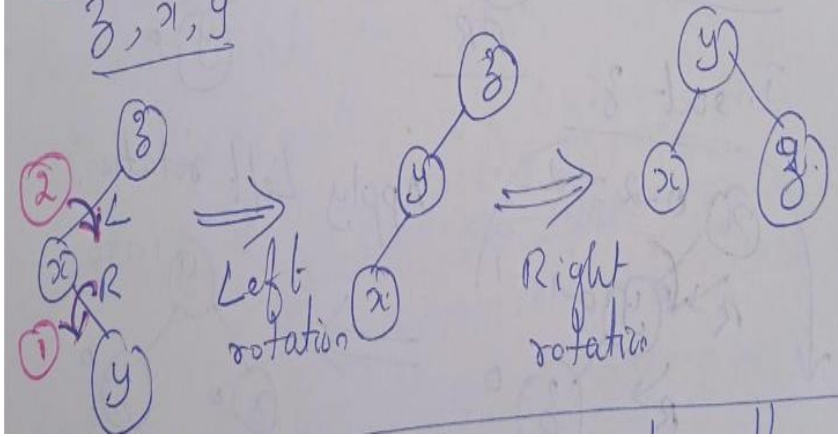


Case 3: x, z, y RL Rotation



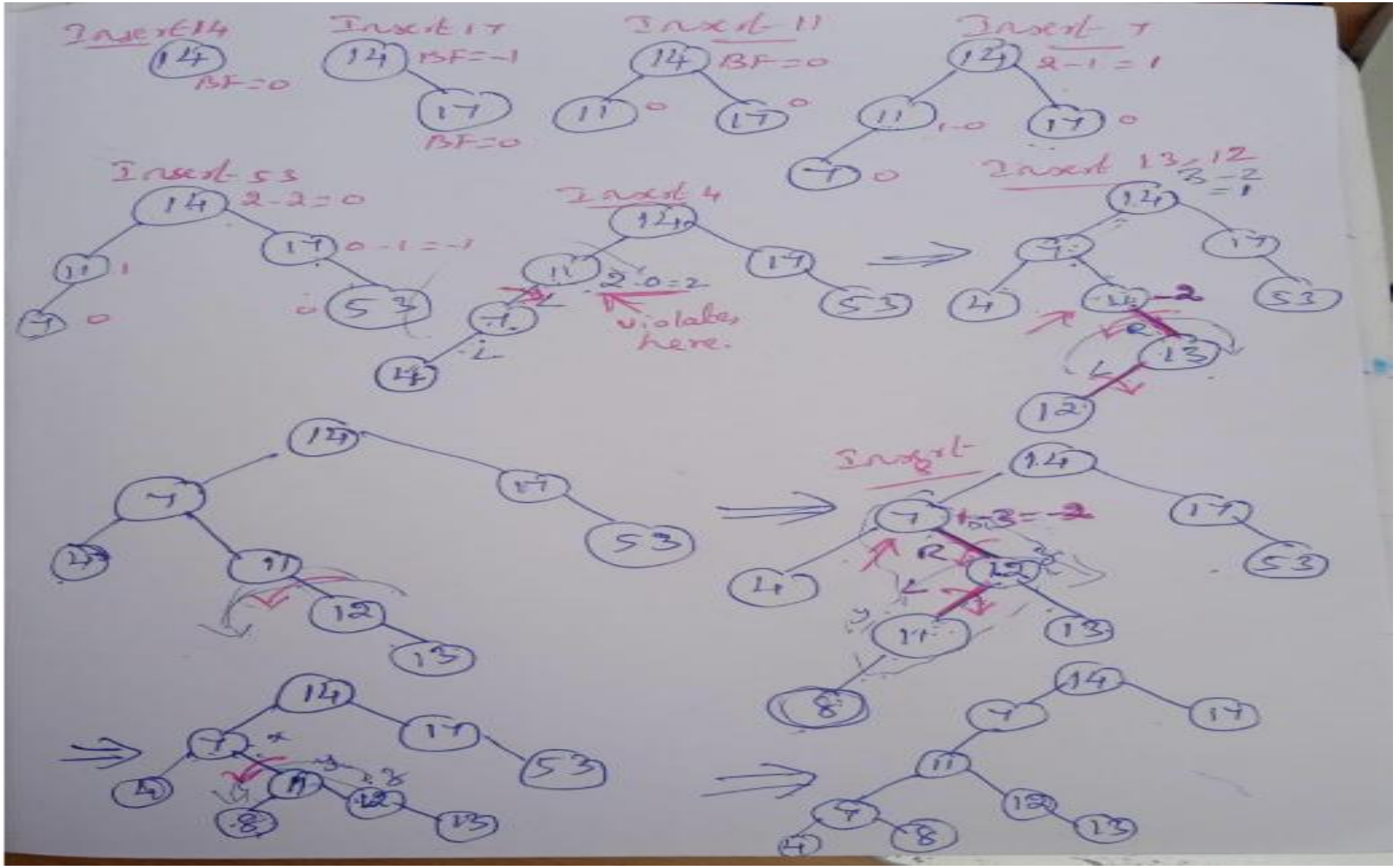
Case 4: LR Rotation.

z, x, y



Construct AVL tree by inserting the following data:

14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20


Insert 14
BF=0
Insert 17
BF=-1
BF=0
Insert 11
BF=0
Insert 7
2-1=1
1-0
Insert 53
2-2=0
0-1=-1
Insert 4
2-0=2
violates here.
Insert 13, 12
3-2=1
Insert
1-0=1
2-0=2
2-1=1

Deletion Operation

- Similar to deletion operation in BST
- Every deletion operation, we need to check with the Balance Factor condition
- If the tree is balanced after deletion go for next operation otherwise perform suitable rotation to make the tree Balanced.

Steps for AVL Tree Deletion

Step 1: Perform BST Deletion: Find and delete the node using standard BST rules:

1. **No Children**: Simply remove the leaf node.
2. **One Child**: Remove the node and link its parent to its single child.
3. **Two Children**: Replace the node with its in-order successor (the smallest node in its right subtree) or in-order predecessor (the largest node in its left subtree), then delete the successor/predecessor from its original position.

Step 2: Update Heights and Check Balance:

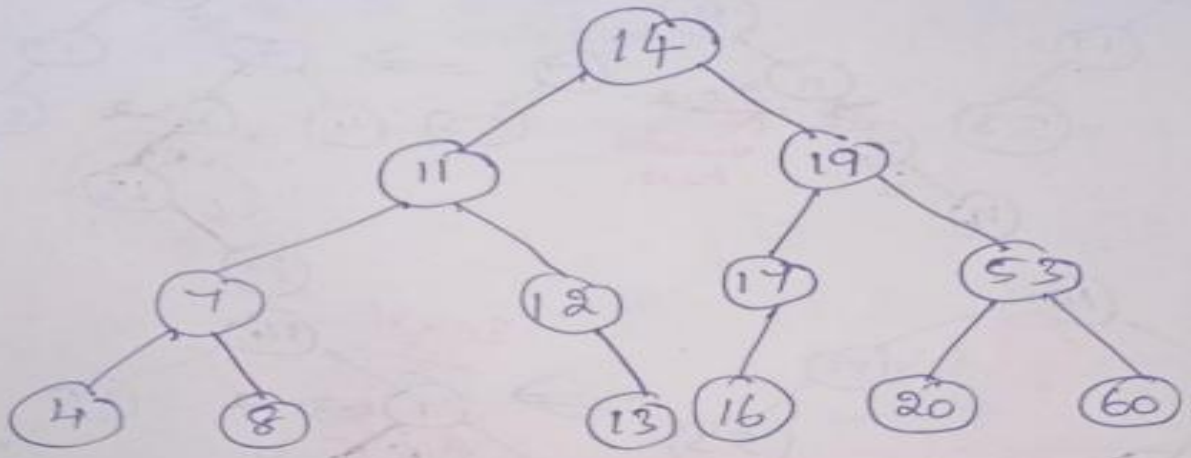
After the node is deleted, traverse upwards from the parent of the deleted node towards the root.

- For each ancestor node, update its height.
- Calculate the balance factor (height of left subtree - height of right subtree) for each node.

Step 3: Perform Rotations (If Necessary):

Deletion in AVL Tree

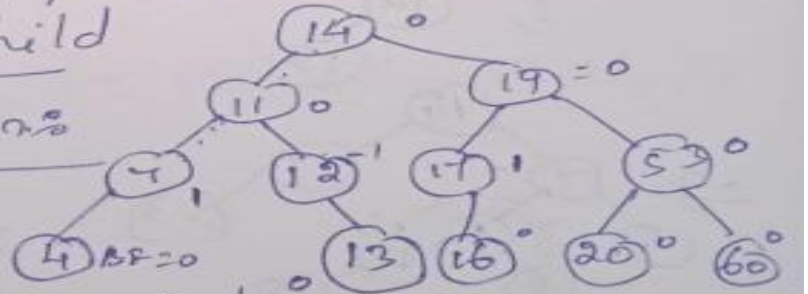
Consider the given AVL tree:



Delete 8, 7, 11, 14 and 17

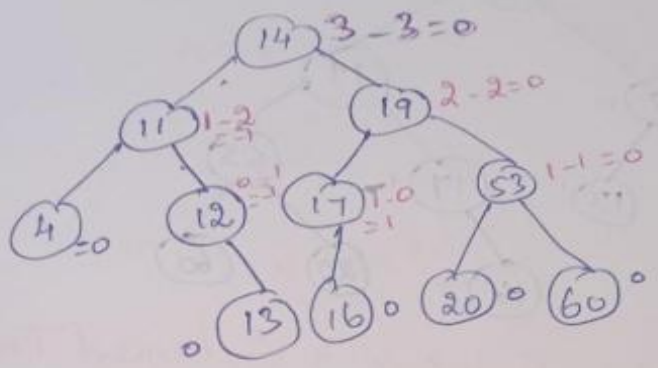
i) Delete 8: No child

After deletion:



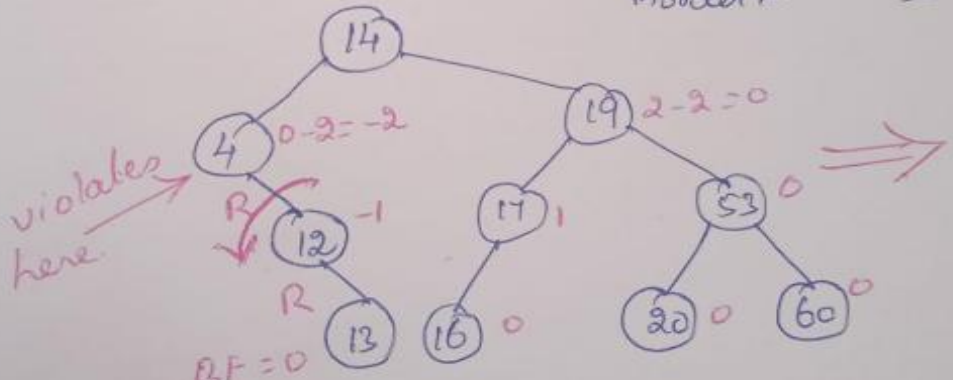
Balanced Tree.

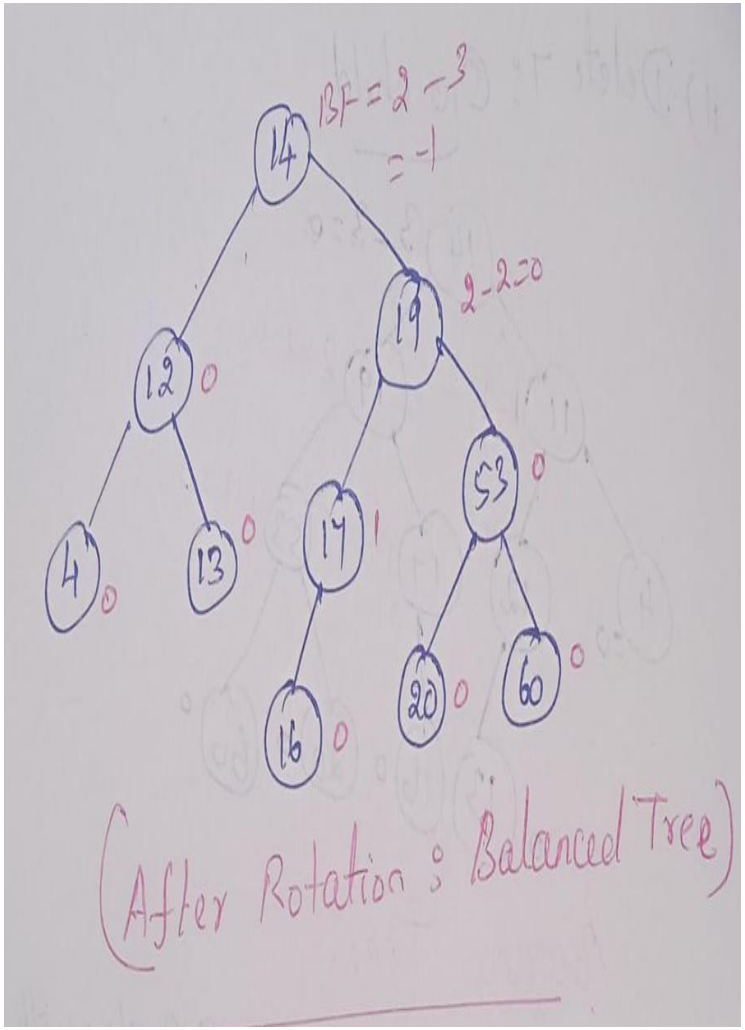
ii) Delete 7: One child



Balance Tree.

Delete 11: (Two children) { Replace with inorder predecessor (or) inorder successor }





(After Rotation; Balanced Tree)

Construct an AVL tree

63,9,19,27,18,108,99,81